Day 152 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 6 reference

Watch: Long Short-Term Memory with PyTorch + Lightning (StatQuest with Josh Starmer) | Multi-Step LSTM Time Series Forecasting (TravelWithDatascientist)

`day152_lstm_forecasting.py`, segment by segment, in the order `main()` uses things. The windows, the network's loss and gradients, Adam training, the recursion, the fit on changes, the forecasts and the control are new, and are copied unchanged into today's exercise file. Day 65's LSTM is imported from Week 10's exercise file, and Days 147, 149 and 151's parts from this week's. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: P = the network's weights (a dictionary) · X = windows, shape (windows, 24, 1) · h_{24} = the last hidden state · F = a fitted forecaster (a dictionary)

Segment 0 — Setup

Imports: another week's folder on the path

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(HERE, "..", "week-10-lstm-gru"))
with contextlib.redirect_stdout(io.StringIO()):
    import day65_exercise as d65
    import day147_exercise as d147
    import day149_exercise as d149
    import day151_exercise as d151
L = 24
HID = 16
SEEDS = (0, 1, 2, 3, 4)
EPOCHS = 300
NVAL = 60
AIRLINE = (0, 1, 0, 1, 12)
```

- `os.path.join(HERE, "..", "week-10-lstm-gru")` — ".." is the parent folder: Week 10's folder sits next to this one.
- `sys.path.insert(0, ...)` — searched first by import, so day65_exercise is found there.
- `with contextlib.redirect_stdout(io.StringIO()):` — anything printed while importing goes into a string and is dropped.
- `SEEDS = (0, 1, 2, 3, 4)` — a tuple: the five random starts.

Segment 1 — The parts

windows and init_params

CODE

```
def windows(z, L):
    X = np.lib.stride_tricks.sliding_window_view(z[:-1], L)[:, :, None].copy()
    return X, z[L:]
def init_params(H, seed, D=1):
    r = np.random.RandomState(seed)
    P = dict(W=r.randn(D + H, 4 * H) / np.sqrt(D + H), b=np.zeros(4 * H), Wy=r.randn(H) * 0.1,
        by=np.zeros(1))
    P["b"][:H] = 1.0
    return P
```

- `sliding_window_view(z[:-1], L)[:, :, None]` — every run of L values (the last value is only ever a target); None adds the input axis.
- `.copy()` — a real array: the window view shares memory with z.
- `dict(W=..., b=..., Wy=..., by=...)` — all the weights in one dictionary, so they can be looped over by name.
- `P["b"][:H] = 1.0` — the first H biases belong to the forget gate (Week 10's order: f, i, g, o).

loss_and_grads and predict

MATH

$$y\text{-}hat = h_{24} W_y + b_y; \quad loss = mean\ (y\text{-}hat - y)^2; \quad dloss/dh_{24} = (2\ (y\text{-}hat - y) / m)\ W_y$$

CODE

```
def loss_and_grads(P, X, Y):
    m, H = len(X), P["Wy"].shape[0]
    z0 = np.zeros((m, H))
    Hs, Cs, gates = d65.lstm_forward(X, z0, z0, P["W"], P["b"])
    hT = Hs[:, -1]
    r = hT @ P["Wy"] + P["by"] - Y
    dy = 2 * r / m
    dW, db = d65.lstm_backward(np.outer(dy, P["Wy"]), X, z0, z0, P["W"], Hs, Cs, gates)
    return np.mean(r ** 2), dict(W=dW, b=db, Wy=hT.T @ dy, by=np.array([dy.sum()]))
def predict(P, X):
    z0 = np.zeros((len(X), P["Wy"].shape[0]))
    return d65.lstm_forward(X, z0, z0, P["W"], P["b"])[0][:, -1] @ P["Wy"] + P["by"]
```

- `d65.lstm_forward(X, z0, z0, P["W"], P["b"])` — Day 65's function, called through its module name.
- `Hs[:, -1]` — every window's hidden state at the last step.
- `np.outer(dy, P["Wy"])` — the gradient at each window's last hidden state: one row per window.
- `np.array([dy.sum()])` — a one-element array, the same shape as by.

adam_train: mini-batches and the best epoch

MATH

$$m \leftarrow 0.9\,m + 0.1\,g;\quad v \leftarrow 0.999\,v + 0.001\,g^2;\quad w \leftarrow w - lr\,(m / (1 - 0.9^k)) / (\sqrt{(v / (1 - 0.999^k))} + 10^{-8})$$

CODE

```
def adam_train(X, Y, Xv, Yv, H, seed, epochs=EPOCHS, lr=0.01, bs=32):
    P = init_params(H, seed); m1 = {k: 0 * v for k, v in P.items()}; m2 = {k: 0 * v for k, v in
        P.items()}
    r, k = np.random.RandomState(seed + 100), 0
    best, hist = (np.inf, None, 0), []
    for ep in range(1, epochs + 1):
        idx = r.permutation(len(X)); tl = 0.0
        for s in range(0, len(X), bs):
            j = idx[s:s + bs]; loss, g = loss_and_grads(P, X[j], Y[j]); k += 1; tl += loss *
                len(j)
            for q in P:
                m1[q] = 0.9 * m1[q] + 0.1 * g[q]; m2[q] = 0.999 * m2[q] + 0.001 * g[q] ** 2
                P[q] -= lr * (m1[q] / (1 - 0.9 ** k)) / (np.sqrt(m2[q] / (1 - 0.999 ** k)) + 1e-8)
        vl = np.mean((predict(P, Xv) - Yv) ** 2); hist.append((tl / len(X), vl))
        if vl < best[0]:
            best = (vl, {q: w.copy() for q, w in P.items()}, ep)
    return best[1], best[2], np.array(hist)
```

- `{k: 0 * v for k, v in P.items()}` — a dictionary comprehension: a zero array shaped like each weight.
- `idx[s:s + bs]` — the next 32 shuffled windows (fewer at the end).
- `m1[q] / (1 - 0.9 ** k)` — Adam's bias correction: the averages start at zero.
- `{q: w.copy() for q, w in P.items()}` — a copy of every weight, so later updates do not change the kept epoch.
- `best[1], best[2]` — indexing the tuple: the kept weights and their epoch.

recursive, fit_lstm and lstm_forecast

CODE

```
def recursive(P, z, L, h):
    zz = list(z[-L:])
    for _ in range(h):
        zz.append(predict(P, np.array(zz[-L:])[None, :, None])[0])
    return np.array(zz[L:])
def fit_lstm(y, L=L, H=HID, seeds=SEEDS, epochs=EPOCHS, nval=NVAL, diff=True):
    d = np.diff(y) if diff else y
    mu, sd = d.mean(), d.std(); z = (d - mu) / sd
    X, Y = windows(z, L)
    fits = [adam_train(X[:-nval], Y[:-nval], X[-nval:], Y[-nval:], H, s, epochs) for s in seeds]
    return dict(models=[f[0] for f in fits], epochs=[f[1] for f in fits], hist=[f[2] for f in
        fits], mu=mu, sd=sd, L=L, diff=diff)
def lstm_forecast(F, y, h):
    d = np.diff(y) if F["diff"] else y
    z = (d - F["mu"]) / F["sd"]
    out = []
    for P in F["models"]:
        f = recursive(P, z, F["L"], h) * F["sd"] + F["mu"]
        out.append(y[-1] + np.cumsum(f) if F["diff"] else f)
    return np.array(out)
```

- `zz = list(z[-L:])` — a Python list, because it grows by one value at a time.
- `np.array(zz[-L:])[None, :, None]` — the newest L values as one window: shape (1, L, 1).
- `[adam_train(...) for s in seeds]` — a list comprehension: one trained network per seed.

- `[f[0] for f in fits]` — the first item of each (weights, epoch, history) tuple.
- `y[-1] + np.cumsum(f) if F["diff"] else f` — a conditional expression: add up the changes, or use the levels as they are.

seasonal_drift: the control

MATH

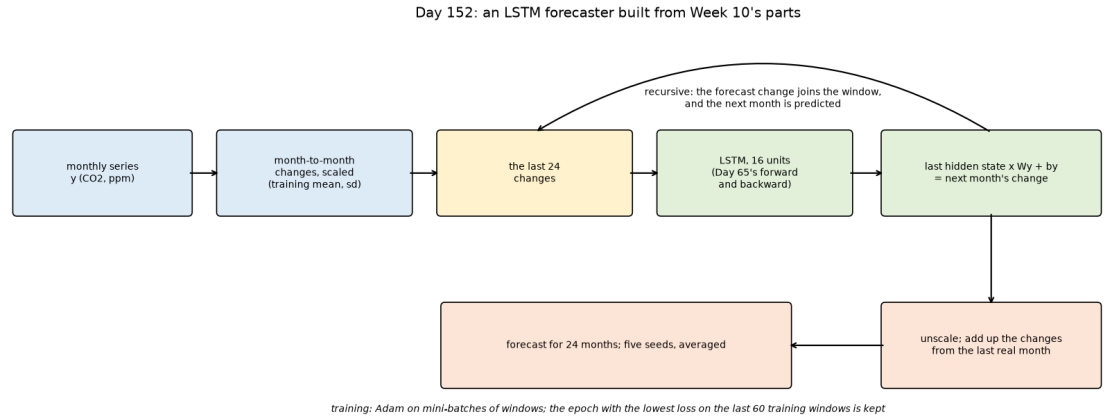
$$y\text{-}hat_{T+h} = y_T + a_{month(T+1)} + \ldots + a_{month(T+h)}$$

CODE

```
def seasonal_drift(y, start, h, years=None):
    d = np.diff(y)
    mo = (np.datetime64(start, "M").astype(int) + np.arange(1, len(y))) % 12
    if years:
        d, mo = d[-12 * years:], mo[-12 * years:]
    avg = np.array([d[mo == k].mean() for k in range(12)])
    fm = (np.datetime64(start, "M").astype(int) + len(y) + np.arange(h)) % 12
    return y[-1] + np.cumsum(avg[fm])
```

- `np.datetime64(start, "M").astype(int)` — the start month as a whole number of months since 1970-01.
- `if years:` — None and 0 are falsy: no window given means all the years.
- `d[mo == k].mean()` — a boolean mask: the changes that fall in calendar month k, averaged.
- `avg[fm]` — fancy indexing: each forecast month's average change.

SYSTEM DIAGRAM



The forecaster built from Week 10's parts.

Segment 2 — The experiments

airline_fit and airline_forecast: fitted once, used many times

CODE

```
def airline_fit(y):
    w = np.diff(y[12:] - y[:-12]); th, s2, _ = d149.fit_css(w, AIRLINE); a, b =
        d149.sarima_polys(th, AIRLINE)
    return d149.full_ar(a), a, b
def airline_forecast(fit, y, h):
    A, a, b = fit; w = np.diff(y[12:] - y[:-12])
    return d149.forecast(y, A, b, d149.css_residuals(w, a, b), h)
```

- `return d149.full_ar(a), a, b` — a tuple of three, unpacked later as A, a, b = fit.
- `d149.css_residuals(w, a, b)` — the errors recomputed on the longer history, with the old coefficients.

demo_levels_vs_changes: the output's hard limit

CODE

```
cap = [(np.abs(P["Wy"]).sum() + P["by"][0]) * lev["sd"] + lev["mu"] for P in lev["models"]]
```

- `np.abs(P["Wy"]).sum() + P["by"][0]` — every |h| < 1, so the output cannot exceed this (in scaled units).
- `* lev["sd"] + lev["mu"]` — back to ppm.

demo_forecast: small functions for the scores

CODE

```
ctl = {yrs: seasonal_drift(yc[:n], tc[0], 24, yrs) for yrs in (None, 10, 5)}
mae = lambda f: np.mean(np.abs(f - act))
bias = lambda f: np.mean(f - act)
```

- `{yrs: ... for yrs in (None, 10, 5)}` — a dictionary keyed by the window: None (all years), 10 or 5.
- `mae = lambda f: ...` — a one-line function; act comes from the enclosing function.
- `map(mae, fc)` — mae applied to each seed's forecast, used inside min() and max().

demo_rolling: the control's averages fixed at 1989-12

CODE

```
mo = (tc[:first].astype("datetime64[M]").astype(int) % 12)[1:]; dd = np.diff(yc[:first])
avg = {k: np.array([v[0][v[1] == j].mean() for j in range(12)]) for k, v in
       (("control, all years", (dd, mo)), ("control, last 10 years", (dd[-120:], mo[-120:])))}
for o in range(first, n - 24 + 1):
    act = yc[o:o + 24]; f = lstm_forecast(F, yc[:o], 24)
    E["LSTM, average of 5"].append(np.abs(f.mean(axis=0) - act)); E["LSTM, single
        seeds"].append(np.abs(f - act).mean(axis=0))
    E["airline ARIMA"].append(np.abs(airline_forecast(ar, yc[:o], 24) - act))
    E["regression, lags 1-13"].append(np.abs(d151.recursive_forecast(yc[:o], tc[0], spec, b, 24) -
        act))
    E["same month last year"].append(np.abs(yc[o - 12:o][np.arange(24) % 12] - act))
    fm = (tc[o - 1].astype("datetime64[M]").astype(int) + 1 + np.arange(24)) % 12
    for k in avg:
        E[k].append(np.abs(yc[o - 1] + np.cumsum(avg[k][fm]) - act))
```

- `(... % 12)[1:]` — the calendar month of each change (the first month has no change).
- `for k, v in ((name, (d, mo)), ...)` — a dictionary comprehension over a tuple of pairs: v[0] the changes, v[1] their months.

- `dd[-120:]` — the last 120 changes: ten years.
- `for k in avg:` — looping over a dictionary gives its keys.

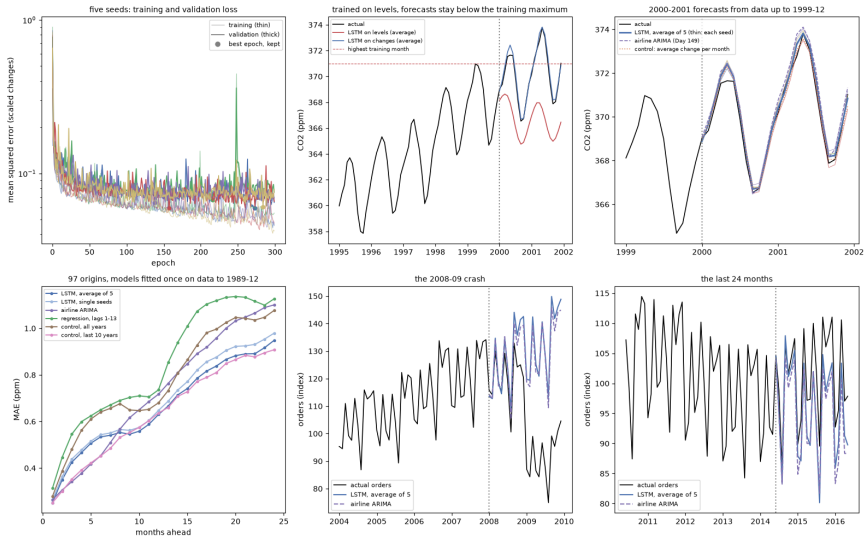
demo_orders: a function of two arguments

CODE

```
mape = lambda f, act: 100 * np.mean(np.abs(np.exp(f - act) - 1))
F = fit_lstm(le[:cut], nval=24); act = le[cut:cut + 24]
```

- `lambda f, act: ...` — a one-line function with two arguments.
- `fit_lstm(le[:cut], nval=24)` — a keyword argument overrides the default 60: a shorter validation block.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `os.path.join(HERE, "..", name)` — a path to a folder next to this one.
- `redirect_stdout(io.StringIO())` — silence printing inside a with block.
- `x[:, :, None]` — add an axis of length 1.
- `{k: f(v) for k, v in d.items()}` — a dictionary comprehension.
- `np.outer(a, b)` — every a[i] times every b[j]: a table.
- `lambda a, b: ...` — a one-line function.
- `map(f, xs)` — f applied to each item, lazily.
- `dict(k=v, ...)` — a dictionary from keyword arguments.

End of Day 152 syntax reference. For the experiments and what they showed, see the Day 152 complete guide.